

TEN ANGRY COMMENTS LATER STORIES ABOUT CODE REVIEWS, TEAMS, AND YESTERDAY'S CODE

JCON 2026 · JOHANNES RABAUER



rabauer.dev/jcon-2026



JOHANNES RABAUER

WORK EXPERIENCE

- Nuclear Power Plant
- Radar Station
- Institute for Information Systems (iisys)

SENIOR SOFTWARE ENGINEER **XDEV**

- Helping out customers in projects
- Open source contributor



SOFTWARE DEVELOPMENT $\neq$ CODING

SOFTWARE DEVELOPMENT $\neq$ CODING



CODING

SOFTWARE DEVELOPMENT $\neq$ CODING



CODING



SPEAKING

SOFTWARE DEVELOPMENT ≠ CODING



CODING



SPEAKING



LISTENING

SOFTWARE DEVELOPMENT ≠ CODING



CODING



SPEAKING



LISTENING



UNDERSTANDING

3

2020 CORONA

2020 CORONA

- Payback application with invoices

2020 CORONA

- Payback application with invoices
- VAT 19% → 16%

2020 CORONA

- Payback application with invoices
- VAT 19% → 16%



Easy, only config value

2020 CORONA

- Payback application with invoices
- VAT 19% → 16%



Which VAT for payback?

2020 CORONA

- Payback application with invoices
- VAT 19% → 16%



Invoices with new legal info

WHY COMMUNICATION IS IMPORTANT



WHY COMMUNICATION IS IMPORTANT



CODE
INSTANTLY

WHY COMMUNICATION IS IMPORTANT



CODE
INSTANTLY



UNDERSTANDING

WHY COMMUNICATION IS IMPORTANT



CODE
INSTANTLY



UNDERSTANDING



RIGHT
PROBLEM

WHY COMMUNICATION IS IMPORTANT



**CODE
INSTANTLY**



UNDERSTANDING



**RIGHT
PROBLEM**



COLLABORATION

3

NEW LIBRARY



CAN I HAZ PEER REVIEW PLZ?

NEW LIBRARY



I'll write my own PR -
with blackjack and hookers

NEW LIBRARY

NEW LIBRARY



NOT
PERFECTION...

NEW LIBRARY



NOT
PERFECTION...



...BUT
LEARNING

NEW LIBRARY



NOT
PERFECTION...



...BUT
LEARNING



PAIR
PROGRAMMING

NEW LIBRARY



NOT
PERFECTION...



...BUT
LEARNING



PAIR
PROGRAMMING

▸ MINUTES ARE
ENOUGH

NEW LIBRARY



NOT
PERFECTION...



...BUT
LEARNING



PAIR
PROGRAMMING

- MINUTES ARE ENOUGH
- SCREENSHARING ENOUGH

3

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



Ingrid

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID

BUG

AUTH-123

MEDIUM

Suppress WARN log for single-use refresh token in OIDC flow

Quarkus OIDC logs a WARN every time a single-use refresh token is consumed. This is expected OIDC spec behaviour, not a bug — the noise pollutes production logs and triggers false alerts in monitoring.

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID

BUG

AUTH-123

MEDIUM

Suppress WARN log for single-use refresh token in OIDC flow

Quarkus OIDC logs a WARN every time a single-use refresh token is consumed. This is expected OIDC spec behaviour, not a bug — the noise pollutes production logs and triggers false alerts in monitoring.

SRC/MAIN/RESOURCES/APPLICATION.PROPERTIES

```
@@ -8,3 +8,7 @@ quarkus.oidc.client-id=portal
quarkus.oidc.application-type=web-app
quarkus.oidc.token.refresh-expired=true
+
+# Refresh tokens are single-use per OIDC spec; suppress only the specific WARN
+# instead of muting all OIDC warnings
+quarkus.log.category."io.quarkus.oidc.runtime.OidcProvider".level=ERROR
```

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID

BUG

AUTH-123

MEDIUM

Suppress WARN log for single-use refresh token in OIDC flow

Quarkus OIDC logs a WARN every time a single-use refresh token is consumed. This is expected OIDC spec behaviour, not a bug — the noise pollutes production logs and triggers false alerts in monitoring.



REVIEWER: NORBERT NEGATIVE

Why are we using Quarkus at all? Spring Security handles OIDC without producing this kind of noise.

SRC/MAIN/RESOURCES/APPLICATION.PROPERTIES

```
@@ -8,3 +8,7 @@ quarkus.oidc.client-id=portal
quarkus.oidc.application-type=web-app
quarkus.oidc.token.refresh-expired=true
+
+# Refresh tokens are single-use per OIDC spec; suppress only the specific WARN
+# instead of muting all OIDC warnings
+quarkus.log.category."io.quarkus.oidc.runtime.OidcProvider".level=ERROR
```

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID

BUG

AUTH-123

MEDIUM

Suppress WARN log for single-use refresh token in OIDC flow

Quarkus OIDC logs a WARN every time a single-use refresh token is consumed. This is expected OIDC spec behaviour, not a bug — the noise pollutes production logs and triggers false alerts in monitoring.

SRC/MAIN/RESOURCES/APPLICATION.PROPERTIES

```
@@ -8,3 +8,7 @@ quarkus.oidc.client-id=portal
  quarkus.oidc.application-type=web-app
  quarkus.oidc.token.refresh-expired=true
+
+# Refresh tokens are single-use per OIDC spec; suppress only the specific WARN
+# instead of muting all OIDC warnings
+quarkus.log.category."io.quarkus.oidc.runtime.OidcProvider".level=ERROR
```



REVIEWER: NORBERT NEGATIVE

Why are we using Quarkus at all? Spring Security handles OIDC without producing this kind of noise.



DEV: INGRID

Quarkus was the team's choice at project start. This warning is standard OIDC spec behaviour — any framework would log it for single-use tokens.

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



CARRY
DECISION
TOGETHER

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



CARRY
DECISION
TOGETHER



EARLY
COOPERATION

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



**CARRY
DECISION
TOGETHER**



**EARLY
COOPERATION**



**LISTEN TO
CONCERNS**

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



CARRY
DECISION
TOGETHER



EARLY
COOPERATION



LISTEN TO
CONCERNS



ARCHITECTURE
DECISION
RECORD

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



**CARRY
DECISION
TOGETHER**



**EARLY
COOPERATION**



**LISTEN TO
CONCERNS**



**ARCHITECTURE
DECISION
RECORD**

► **DOCUMENT**

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



**CARRY
DECISION
TOGETHER**



**EARLY
COOPERATION**



**LISTEN TO
CONCERNS**



**ARCHITECTURE
DECISION
RECORD**

- ▶ **DOCUMENT**
- ▶ **IN CODE**

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



**CARRY
DECISION
TOGETHER**



**EARLY
COOPERATION**



**LISTEN TO
CONCERNS**



**ARCHITECTURE
DECISION
RECORD**

- ▶ **DOCUMENT**
- ▶ **IN CODE**
- ▶ **MARKDOWN**

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



**CARRY
DECISION
TOGETHER**



**EARLY
COOPERATION**



**LISTEN TO
CONCERNS**



**ARCHITECTURE
DECISION
RECORD**

- ▶ **DOCUMENT**
- ▶ **IN CODE**
- ▶ **MARKDOWN**
- ▶ **USE TEMPLATE**

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS. - ADR

TEMPLATE BY MICHAEL NYGARD

```
# Title
## Status
What is the status, such as proposed, accepted, rejected,
deprecated, superseded, etc.?

## Context
What is the issue that we're seeing that is motivating this
decision or change?

## Decision
What is the change that we're proposing and/or doing?

## Consequences
What becomes easier or more difficult to do because of this
change?
```

QUARKUS DECISION EXAMPLE

```
# Adopt Quarkus Instead of Spring Boot for Microservice
## Status
Accepted
## Context
The project requires building cloud-native microservices
with fast startup times, low memory consumption, and high
runtime performance.

...

## Decision
We decided to use Quarkus as the primary framework for
developing the projects microservices instead of Spring
Boot.

This decision is based on the following key factors...

...

## Consequences
### Positive
* Reduced resource usage (CPU and memory), leading to lower
infrastructure costs...

### Negative
* Smaller ecosystem compared to Spring Boot, which may
limit available integrations...
```

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID



NORBERT NEGATIVE

THIS LIBRARY IS TRASH. MAINTAINERS ARE IDIOTS.



INGRID



NORBERT NEGATIVE

3

PR PING PONG



Herbert

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.



REVIEWER: MONA MALICIOUS

Changing this to partial saves breaks data consistency. If one appointment fails, others sharing the same room or instructor could end up in a conflicted state.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```



REVIEWER: MONA MALICIOUS

Changing this to partial saves breaks data consistency. If one appointment fails, others sharing the same room or instructor could end up in a conflicted state.



DEV: HERBERT

That is the business requirement though. Right now users lose all five appointments because just one has a time conflict. The product team explicitly wants partial success.

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

breaks data consistency. If one appointment fails, others sharing the same room or instructor could end up in a conflicted state.



DEV: HERBERT

That is the business requirement though. Right now users lose all five appointments because just one has a time conflict. The product team explicitly wants partial success.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```



REVIEWER: MONA MALICIOUS

What about confirmation emails? They fire after `saveAll` returns. With partial saves you'd send confirmations for some but not others — that logic does not exist yet.

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

though. Right now users lose all five appointments because just one has a time conflict. The product team explicitly wants partial success.



REVIEWER: MONA MALICIOUS

What about confirmation emails? They fire after `saveAll` returns. With partial saves you'd send confirmations for some but not others — that logic does not exist yet.



DEV: HERBERT

`SaveResult` already carries per-appointment status. The email service can iterate that and skip failed entries. I checked — it is a small change on that side.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```

I may tire after `saveAll` returns. With partial saves you'd send confirmations for some but not others — that logic does not exist yet.



DEV: HERBERT

`SaveResult` already carries per-appointment status. The email service can iterate that and skip failed entries. I checked — it is a small change on that side.



REVIEWER: MONA MALICIOUS

I am still not comfortable. If two appointments share a linked resource — same instructor, same room — saving only one could violate constraints we enforce at the service layer, not just the DB.

PR PING PONG



HERBERT

FEATURE

CAL-412

MEDIUM

Save appointments individually — skip failed, keep successful

Currently `saveAll()` wraps all appointments in a single transaction: if one save fails, every appointment is rolled back. Change the behaviour so each appointment is saved independently — failed ones are skipped and the rest are persisted.

appointment status. The email service can iterate that and skip failed entries. I checked — it is a small change on that side.



REVIEWER: MONA MALICIOUS

I am still not comfortable. If two appointments share a linked resource — same instructor, same room — saving only one could violate constraints we enforce at the service layer, not just the DB.

SRC/CALENDAR/APPOINTMENTSERVICE.JAVA

```
@@ -24,9 +24,9 @@ public class AppointmentService {  
    // ...  
}
```



DEV: HERBERT

Those constraints are real and I don't have a clean answer for the linked-resource case yet. I think we need a team session to map the actual dependencies before anyone can implement this safely.

PR PING PONG

PR PING PONG



WRITE
COMMENTS

PR PING PONG



WRITE
COMMENTS



TALK

PR PING PONG



WRITE
COMMENTS



TALK

▸ IN PERSON

PR PING PONG



WRITE
COMMENTS



TALK

- IN PERSON
- VIDEO CALL

PR PING PONG



WRITE
COMMENTS



TALK

- IN PERSON
- VIDEO CALL
- AUDIO CALL

PR PING PONG



WRITE
COMMENTS



TALK

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

- ▶ REV: COMMENT

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

- ▶ REV: COMMENT
- ▶ DEV: ANSWERS

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

- ▶ REV: COMMENT
- ▶ DEV: ANSWERS
- ▶ REV: ANSWERS

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

- ▶ REV: COMMENT
- ▶ DEV: ANSWERS
- ▶ REV: ANSWERS
- ▶ DEV: ANSWERS

PR PING PONG



WRITE
COMMENTS



TALK



2 LOOP RULE

- ▶ IN PERSON
- ▶ VIDEO CALL
- ▶ AUDIO CALL
- ▶ CHAT

- ▶ REV: COMMENT
- ▶ DEV: ANSWERS
- ▶ REV: ANSWERS
- ▶ DEV: ANSWERS
- ▶ TALK

PR PING PONG



HERBERT



MONA MALICIOUS

PR PING PONG



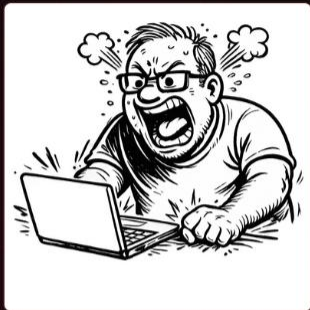
HERBERT



MONA MALICIOUS

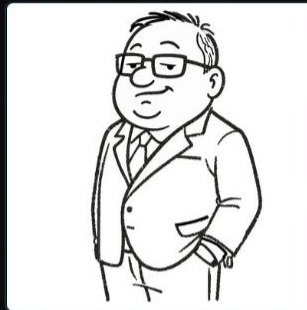
3

EMOTION 👍



/**

```
* This f**ing function was changed for the
* third time, and i am sick and tired. DO
* NOT CHANGE THIS FUNCTION!
* Jamie AND Laura tried over and over and
* they FAILED! Changing this function
* does NOT optimize anything!
* Just leave it.
*/
```



/**

```
* This function has been modified multiple
* times without improving performance.
* Do not change it unless you have clear
* evidence of a measurable benefit and
* have validated the impact with
* proper testing.
*/
```

EMOTION 👎 - EXAMPLES

EMOTION 👎 - EXAMPLES



REVIEWER: MONA MALICIOUS

Every time I review your PRs, I have to rewrite half of the approach. Please learn the project basics before submitting again.

EMOTION 👎 - EXAMPLES



REVIEWER: MONA MALICIOUS

Every time I review your PRs, I have to rewrite half of the approach. Please learn the project basics before submitting again.



REVIEWER: NORBERT NEGATIVE

This implementation is chaos. It looks like you coded first and thought later.

EMOTION 👉 - EXAMPLES



REVIEWER: MONA MALICIOUS

Every time I review your PRs, I have to rewrite half of the approach. Please learn the project basics before submitting again.



REVIEWER: NORBERT NEGATIVE

This implementation is chaos. It looks like you coded first and thought later.



REVIEWER: MONA MALICIOUS

You ignored previous feedback again. At this point, I am not sure you are listening to the team at all.

EMOTION 👉 - EXAMPLES



REVIEWER: MONA MALICIOUS

Every time I review your PRs, I have to rewrite half of the approach. Please learn the project basics before submitting again.



REVIEWER: MONA MALICIOUS

You ignored previous feedback again. At this point, I am not sure you are listening to the team at all.



REVIEWER: NORBERT NEGATIVE

This implementation is chaos. It looks like you coded first and thought later.



REVIEWER: NORBERT NEGATIVE

If this is your best version after multiple attempts, we have a serious quality problem.

EMOTION - FEEDBACK

EMOTION - FEEDBACK



**BE
CONSTRUCTIVE**

EMOTION - FEEDBACK



**BE
CONSTRUCTIVE**



VALUE WORK

3



CUSTOM XML INSTEAD OF STANDARDS

✗ VAADIN-SECURITY.XML

```
<?xml version="1.0" encoding="UTF-8"?>
<vaadin-security>
  <view-mappings>

    <view path="/settings"
          class="... SettingsView">
      <access>
        <require-role>ADMIN</require-role>
      </access>
    </view>

    <view path="/info"
          class="... InfoView">
      <access>
        <require-role>ADMIN</require-role>
        <require-role>USER</require-role>
      </access>
    </view>

    <view path="/dashboard"
          class="... DashboardView">
```

✓ STANDARD VAADIN

```
@Route("settings")
@RolesAllowed("ADMIN")
public class SettingsView
    extends VerticalLayout {}

@Route("info")
@RolesAllowed({"ADMIN", "USER"})
public class InfoView
    extends VerticalLayout {}

@Route("dashboard")
@PermitAll
public class DashboardView
    extends VerticalLayout {}
```

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?



LESS BUGS

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?



LESS BUGS



**LESS
DUPLICATES**

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?



LESS BUGS



**LESS
DUPLICATES**



**LESS
COMPLEXITY**

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?



LESS BUGS



**LESS
DUPLICATES**



**LESS
COMPLEXITY**



**LONGER
SUPPORT**

CUSTOM XML INSTEAD OF STANDARDS - WHY STANDARDS?



LESS BUGS



LESS



LONGER
SUPPORT



CUSTOM XML INSTEAD OF STANDARDS

✓ STANDARD VAADIN

```
@Route("settings")
@RolesAllowed("ADMIN")
public class SettingsView
    extends VerticalLayout {}

@Route("info")
@RolesAllowed({"ADMIN", "USER"})
public class InfoView
    extends VerticalLayout {}

@Route("dashboard")
@PermitAll
public class DashboardView
    extends VerticalLayout {}
```

VAADIN-ROUTES-EXPORT-MAVEN-PLUGIN

```
<routes>
  <route>
    <path>settings</path>
    <className>... SettingsView</className>
    <layouts>
      <layout>... MainLayout</layout>
    </layouts>
    <roles>
      <role>ADMIN</role>
    </roles>
    <access>RESTRICTED</access>
    <securitySource>ANNOTATION</securitySource>
  </route>
  <route>
    <path>info</path>
    <className>... InfoView</className>
    ...
  </route>
</routes>
```

LEGACY



LEGACY

LEGACY



VALUE WORK

LEGACY



VALUE WORK



SAFETY FIRST

LEGACY



VALUE WORK



SAFETY FIRST

▸ TESTS

LEGACY



VALUE WORK



SAFETY FIRST

- ▶ TESTS
- ▶ PROTOTYPES

3

...COULD YOU FIX THAT TOO?



Yuan

...COULD YOU FIX THAT TOO?



YUAN

BUG

CORE-224

HIGH

NullPointerException in OrderService.process() for empty cart

Calling OrderService.process() with a null or empty items list throws an unhandled NullPointerException. Adding an explicit guard clause at the entry point.

...COULD YOU FIX THAT TOO?



YUAN

BUG

CORE-224

HIGH

NullPointerException in OrderService.process() for empty cart

Calling OrderService.process() with a null or empty items list throws an unhandled NullPointerException. Adding an explicit guard clause at the entry point.

SRC/CORE/ORDERSERVICE.JAVA

```
@@ -44,6 +44,9 @@ public class OrderService {
    public Order process(List<OrderItem> items) {
+       if (items == null || items.isEmpty()) {
+           throw new IllegalArgumentException("Cart must not be empty");
+       }
        for (OrderItem item : items) {
```

...COULD YOU FIX THAT TOO?



YUAN

BUG

CORE-224

HIGH

NullPointerException in OrderService.process() for empty cart

Calling OrderService.process() with a null or empty items list throws an unhandled NullPointerException. Adding an explicit guard clause at the entry point.



REVIEWER: NORBERT NEGATIVE

While you're in here, can you also fix the deprecated Date usage on line 12? It's been bothering me for a while.

SRC/CORE/ORDERSERVICE.JAVA

```
@ -44,6 +44,9 @@ public class OrderService {  
    public Order process(List<OrderItem> items) {  
+        if (items == null || items.isEmpty()) {  
+            throw new IllegalArgumentException("Cart must not be empty");  
+        }  
        for (OrderItem item : items) {
```

...COULD YOU FIX THAT TOO?



YUAN

BUG

CORE-224

HIGH

NullPointerException in OrderService.process() for empty cart

Calling OrderService.process() with a null or empty items list throws an unhandled NullPointerException. Adding an explicit guard clause at the entry point.

SRC/CORE/ORDERSERVICE.JAVA

```
@@ -44,6 +44,9 @@ public class OrderService {
    public Order process(List<OrderItem> items) {
+       if (items == null || items.isEmpty()) {
+           throw new IllegalArgumentException("Cart must not be empty");
+       }
        for (OrderItem item : items) {
```



REVIEWER: NORBERT NEGATIVE

While you're in here, can you also fix the deprecated Date usage on line 12? It's been bothering me for a while.



DEV: YUAN

That's unrelated to this fix. I'd rather keep the PR focused and not risk introducing new bugs.

...COULD YOU FIX THAT TOO?

...COULD YOU FIX THAT TOO?



BOY SCOUT
RULE

...COULD YOU FIX THAT TOO?



BOY SCOUT
RULE

▶ LEGACY
SYSTEM

...COULD YOU FIX THAT TOO?



BOY SCOUT
RULE

- ▶ LEGACY
SYSTEM
- ▶ SMALL SCOPES

...COULD YOU FIX THAT TOO?



BOY SCOUT
RULE



KEEP PR
FOCUSED

- ▶ LEGACY
SYSTEM
- ▶ SMALL SCOPES

...COULD YOU FIX THAT TOO?



BOY SCOUT
RULE

- ▶ LEGACY
SYSTEM
- ▶ SMALL SCOPES



KEEP PR
FOCUSED

- ▶ HUGE CHANGES

...COULD YOU FIX THAT TOO?



**BOY SCOUT
RULE**

- ▶ **LEGACY
SYSTEM**
- ▶ **SMALL SCOPES**



**KEEP PR
FOCUSED**

- ▶ **HUGE CHANGES**
- ▶ **CREATE A
FOLLOW-UP
TICKET**

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

▶ **SMALL**

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ **SMALL**
- ▶ **EASY TO GRASP**

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY



DON'T CHANGE

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY



DON'T CHANGE

- ▶ TOO MANY CHANGES

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY



DON'T CHANGE

- ▶ TOO MANY CHANGES
- ▶ ALREADY TOO BIG

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES

CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY

DON'T CHANGE

- ▶ TOO MANY CHANGES
- ▶ ALREADY TOO BIG
- ▶ CONTEXT IS UNCLEAR

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY



DON'T CHANGE

- ▶ TOO MANY CHANGES
- ▶ ALREADY TOO BIG
- ▶ CONTEXT IS UNCLEAR
- ▶ FRAGILE HOTSPOT

...COULD YOU FIX THAT TOO? - UNIVERSAL RULES



CHANGE

- ▶ SMALL
- ▶ EASY TO GRASP
- ▶ LOCAL RISK ONLY
- ▶ EASY TO VERIFY
- ▶ DIRECTLY RELATED
- ▶ INCREASES READABILITY



DON'T CHANGE

- ▶ TOO MANY CHANGES
- ▶ ALREADY TOO BIG
- ▶ CONTEXT IS UNCLEAR
- ▶ FRAGILE HOTSPOT
- ▶ NEEDS TEAM AGREEMENT

...COULD YOU FIX THAT TOO?



YUAN



NORBERT NEGATIVE

...COULD YOU FIX THAT TOO?



YUAN



NORBERT NEGATIVE

3

AI

AI



COMMUNICATION
IMPORTANT

AI



COMMUNICATION
IMPORTANT



BE DIRECT

AI



COMMUNICATION
IMPORTANT



BE DIRECT



BE SPECIFIC
(ABOUT
TECHNOLOGY)

AI – FURTHER READING

AI – FURTHER READING

[GITHUB.COM/JOELPARKERHENDERSON/ARCHITECTURE-
DECISION-RECORD](https://github.com/joelparkerhenderson/architecture-decision-record)

README Code of conduct Issues

Architecture decision record (ADR)

An architecture decision record (ADR) is a document that captures an important architecture decision made along with its context and consequences.

Contents

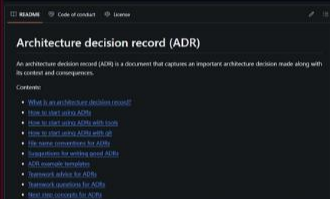
- When to use an Architecture Decision Record
- How to create an ADR
- How to create an ADR with tools
- How to create an ADR with AI
- The ADR template for AI
- Examples for writing an ADR
- ADR template examples
- Template advice for ADRs
- Template examples for ADRs
- How to compare ADRs

Architecture Decision Records

A practical ADR reference with templates and examples for documenting team decisions.

AI – FURTHER READING

[GITHUB.COM/JOELPARKERHENDERSON/ARCHITECTURE-
DECISION-RECORD](https://github.com/joelparkerhenderson/architecture-decision-record)



Architecture Decision Records

A practical ADR reference with templates and examples for documenting team decisions.

[LLM-CODING.GITHUB.IO/SEMANTIC-ANCHORS](https://llm-coding.github.io/semantic-anchors)

LLM

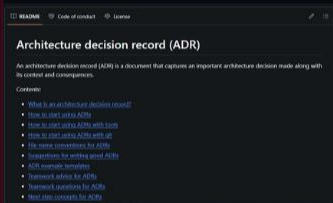


Semantic Anchors

Semantic Anchors

A catalog of concrete prompting concepts that helps teams describe review and implementation patterns more precisely.

AI – FURTHER READING



Architecture Decision Records

A practical ADR reference with templates and examples for documenting team decisions.

LLM-CODING.GITHUB.IO/SEMANTIC-ANCHORS

LLM



Semantic Anchors

A catalog of concrete prompting concepts that helps teams describe review and implementation patterns more precisely.

AI4JVM.COM

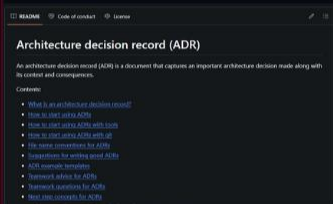
JVM



AI4JVM

A curated starting point for JVM-focused AI tooling, from coding assistants to agent and inference frameworks.

AI – FURTHER READING



Architecture Decision Records

A practical ADR reference with templates and examples for documenting team decisions.

LLM-CODING.GITHUB.IO/SEMANTIC-ANCHORS

LLM



Semantic Anchors

A catalog of concrete prompting concepts that helps teams describe review and implementation patterns more precisely.

AI4JVM.COM

JVM



AI4JVM

A curated starting point for JVM-focused AI tooling, from coding assistants to agent and inference frameworks.

RABAUER.DEV/BLOG

Blog



Live-Coding Learnings

Short write-ups from real Java and AI sessions, focused on usable patterns instead of abstract hype.

3

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM



RESPECT EACH OTHERS
WORK

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM



**RESPECT EACH OTHERS
WORK**



USE STANDARDS

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



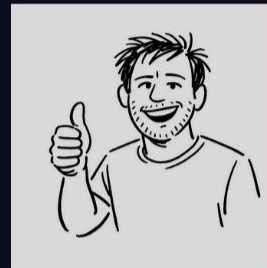
RIGHT PROBLEM



RESPECT EACH OTHERS
WORK



USE STANDARDS



CONSIDER TALKING
INSTEAD OF WRITING

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM



RESPECT EACH OTHERS
WORK



USE STANDARDS



CONSIDER TALKING
INSTEAD OF WRITING



CARRY DECISION
TOGETHER

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM



**RESPECT EACH OTHERS
WORK**



USE STANDARDS



**CONSIDER TALKING
INSTEAD OF WRITING**



**CARRY DECISION
TOGETHER**



**REMOVE EGO & STAY
CONSTRUCTIVE**

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



RIGHT PROBLEM



RESPECT EACH OTHERS
WORK



USE STANDARDS



CONSIDER TALKING
INSTEAD OF WRITING



CARRY DECISION
TOGETHER



REMOVE EGO & STAY
CONSTRUCTIVE



DEFINE ISSUE SCOPE

PRACTICAL REVIEW FRAMEWORK FOR MATURE TEAMS



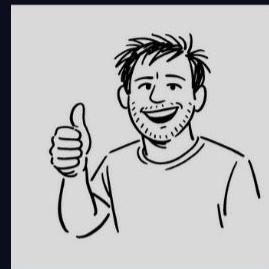
RIGHT PROBLEM



**RESPECT EACH OTHERS
WORK**



USE STANDARDS



**CONSIDER TALKING
INSTEAD OF WRITING**



**CARRY DECISION
TOGETHER**



**REMOVE EGO & STAY
CONSTRUCTIVE**



DEFINE ISSUE SCOPE



PAIR INSTEAD OF REWRITE

THANK YOU



rabauer.dev/jcon-2026

QUESTIONS WITH SLIDO

FEEDBACK

SLIDES

LINKS

CONTACT

THANK YOU



rabauer.dev/jcon-2026

QUESTIONS WITH SLIDO

FEEDBACK

SLIDES

LINKS

CONTACT



COMMIT CARDS V1.1

THANK YOU



rabauer.dev/jcon-2026

QUESTIONS WITH SLIDO

FEEDBACK

SLIDES

LINKS

CONTACT



COMMIT CARDS V1.1



BOOTH